Decodificación de audio con FFmpeg

Este programa en C++ decodifica un archivo MP3 y guarda el resultado como audio crudo (PCM). Está basado en el ejemplo `decode_audio.c` de FFmpeg y usa directamente la biblioteca libavcodec de FFmpeg 9.0.2. Se ejecuta indicando el archivo de entrada y el de salida: `decodificacion.exe <entrada> <salida>`.

Al iniciar, `main()` reserva el paquete con `av_packet_alloc()`, busca el decodificador con `avcodec_find_decoder(AV_CODEC_ID_MP3)`, crea el parser con `av_parser_init()` y prepara el contexto del codec con `avcodec_alloc_context3()` y `avcodec_open2()`. Después abre ambos archivos y llama a `skip_id3v2_tag()`.

La función `skip_id3v2_tag()` salta la etiqueta de metadatos ID3 que puede haber al inicio del MP3. Es necesaria porque el programa lee el archivo directamente, sin libavformat, y esos bytes no son audio: si llegan al decodificador, este falla con el error *Header missing*.

El archivo se lee por bloques de hasta 20 480 bytes. En cada vuelta del ciclo principal, `av_parser_parse2()` separa los datos en frames de audio y cada frame completo se envía a `decode()`. Cuando quedan menos de 4096 bytes sin procesar, se mueven al inicio del búfer y se completa con más datos del archivo.

La función `decode()` entrega el paquete con `avcodec_send_packet()` y recibe el audio con `avcodec_receive_frame()`. Cada muestra se escribe en el archivo de salida, alternando los canales si hay más de uno. Al terminar el archivo se llama a `decode()` con un paquete vacío para obtener lo que aún quede en el decodificador.

Por último, `get_format_from_sample_fmt()` obtiene el nombre del formato de las muestras y el programa imprime el comando `ffplay` para reproducir la salida. El PCM no tiene cabecera, así que el formato, los canales y la frecuencia deben indicarse a mano.

Respecto al ejemplo original se hicieron dos cambios: el decodificador pasó de `AV_CODEC_ID_MP2` a `AV_CODEC_ID_MP3` y se agregó `skip_id3v2_tag()`. El programa solo admite audio MPEG; otros formatos, como AAC o WAV, requerirían libavformat.

La primera captura muestra la decodificación de `tono.mp3`. El resultado es en punto flotante de 32 bits (`f32le`), mono, a 44 100 Hz. El archivo generado pesa 543 744 bytes, es decir, 135 936 muestras de 4 bytes o unos 3,08 segundos de audio. El aviso sobre el formato planar viene del ejemplo original y aquí no afecta, porque el audio tiene un solo canal.



La segunda captura muestra la conversión del PCM a WAV con `ffmpeg`, para poder escucharlo en un reproductor común. Los parámetros `-f f32le -ac 1 -ar 44100` describen el PCM de entrada, y FFmpeg lo guarda en WAV con muestras de 16 bits (`pcm_s16le`). La duración de 3,08 segundos coincide con el cálculo anterior.

